



EULLER LUCAS MENDES ALVES 2022008493

LUCAS APARECIDO SILVINO TOLEDO

2022002767

PEDRO HENRIQUE DOS SANTOS BARBOSA 2022013760

CLASSIFICAÇÃO DE ESPÉCIES DE PÁSSAROS ATRAVÉS DE SONS

ITAJUBÁ 2025

LISTA DE ILUSTRAÇÕES

Figura 1: Diagrama de blocos do sistema a ser implementado	6
Figura 2: Placa de desenvolvimento	7
Figura 3: Parâmetros MFE	11
Figura 4: Parâmetros padrão do classificador no Edge Impulse.	12
Figura 5: Classificador com o número de filtros modificado.	12
Figura 6: função Turn_off_leds	14
Figura 7: função light_led_by_index	14
Figura 8: Utilização da função light_led_by_index	15

SUMÁRIO

1 INTRODUÇÃO.....	4
2 OBJETIVOS PROPOSTOS E ESPECÍFICOS	5
2.1 Objetivos Gerais	5
2.2 Objetivos Específicos	5
3 DESENVOLVIMENTO.....	6
3.1 Hardware utilizado.....	6
3.2 Aquisição e Preparação de Dados para Modelo de Rede Neural	7
3.2.1 Coleta de Dados	7
3.2.2 Padronização e Processamento de Áudio.....	8
3.2.3 A Importância da Curadoria de Dados: Uma Lição Prática.....	9
3.3 Processamento e Estruturação do Dataset.....	10
3.4 Composição Final do Dataset	10
3.5 Pré processamento	10
3.6 Projeto do modelo.....	11
3.7 Inferência.....	13
4 CONCLUSÃO.....	16
REFERÊNCIAS	17
APÊNDICE A – CONTEÚDOS DO PROJETO	18

1 INTRODUÇÃO

Nos últimos anos, o avanço da inteligência artificial e a popularização do aprendizado de máquina embarcado (TinyML) têm permitido o desenvolvimento de soluções inteligentes em dispositivos de baixo custo e consumo energético, como o Arduino Nano 33 BLE Sense. Esses avanços possibilitam tarefas complexas, como classificação de padrões, diretamente na borda (edge), sem necessidade de conexão com a nuvem.

Este projeto visa desenvolver um sistema de classificação de sons de pássaros utilizando aprendizado de máquina implementado no Arduino Nano 33 BLE Sense, com inferência em tempo real. Para isso, foram usadas a plataforma Edge Impulse para coleta de dados, extração de características, treinamento e geração do modelo, e a Arduino IDE para o deploy do firmware no Arduino.

Embora a proposta inicial contemplasse dez espécies, o escopo foi reduzido para quatro classes: Corvo, Tico-Tico, Trinca-Ferro e sons ambientes. Para facilitar a visualização dos resultados pelos LEDs do Arduino. Os dados, incluem 63 amostras de 2 segundos para cada espécie de pássaros e 60 para sons ambientes, divididos em 79% para treinamento e 21% para teste.

O pré-processamento utilizou a feature Mel-Frequency Energy (MFE), mais adequada para sons não humanos que o tradicional Mel-Frequency Cepstral Coefficients (MFCC). A saída do MFE alimentou um modelo de rede neural convolucional com duas camadas, treinado no Edge Impulse.

O modelo alcançou 97,3% de acurácia no treinamento e 96,72% nos testes, comprovando boa capacidade de generalização. Com esses resultados, o modelo foi implementado no Arduino, utilizando seu microfone digital integrado para classificação sonora em tempo real. O projeto aplica na prática os conceitos da disciplina, demonstrando a viabilidade da inteligência artificial na borda em dispositivos embarcados de baixo custo e consumo e poucos recursos de memória disponíveis.

2 OBJETIVOS PROPOSTOS E ESPECÍFICOS

2.1 Objetivos Gerais

O presente relatório tem como objetivo principal o desenvolvimento de um sistema classificador capaz de identificar espécies de pássaros e diferenciá-las de sons ambientes. Para isso, utiliza-se a plataforma Edge Impulse para treinamento do modelo de aprendizado de máquina, com posterior implementação embarcada (deploy) no microcontrolador Arduino Nano 33 BLE Sense, permitindo a classificação sonora em tempo real.

2.2 Objetivos Específicos

- 1) Realizar o pré-processamento de dados de áudio utilizando a técnica Mel-Frequency Energy (MFE), adequada para reconhecimento de sons naturais em aplicações embarcadas.
- 2) Projetar, treinar e avaliar um modelo de rede neural convolucional (CNN) para a classificação de três espécies de pássaros e sons ambientes, por meio da plataforma Edge Impulse.
- 3) Realizar o deploy do modelo treinado no microcontrolador Arduino Nano 33 BLE Sense, integrando captura de áudio para a inferência do modelo em tempo real.
- 4) Indicar por meio dos LEDs RGB e BUILTIN qual o resultado da classificação.

3 DESENVOLVIMENTO

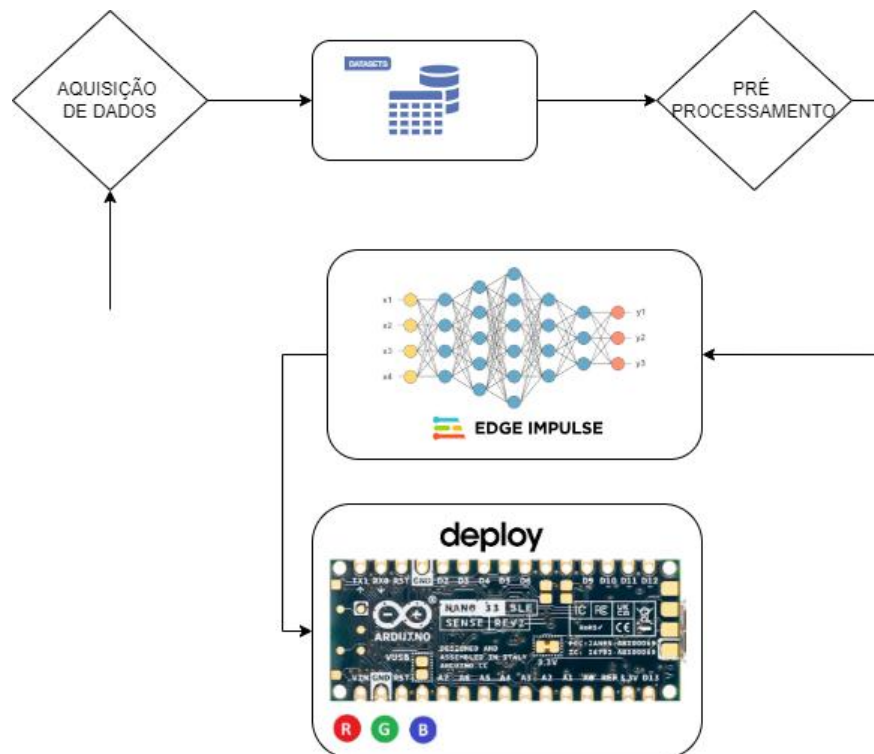


Figura 1: Diagrama de blocos do sistema a ser implementado

3.1 Hardware utilizado

O projeto utiliza como plataforma principal o Arduino Nano 33 BLE Sense, presente no Tiny Machine Learning Kit da Arduino, disponibilizado pela UNIFEI. Trata-se de uma placa compacta, desenvolvida especialmente para aplicações de aprendizado de máquina embarcado (TinyML). É equipada com o microcontrolador nRF52840 (ARM Cortex-M4, 32 bits, 64 MHz), 1 MB de memória Flash e 256 KB de RAM, tornando-a adequada para a execução local de modelos leves de inferência diretamente no dispositivo.

Um dos principais diferenciais desta placa é a presença de sensores embarcados integrados, incluindo:

- Microfone digital (MP34DT05) – sensor utilizado neste projeto, responsável pela captura dos sons dos pássaros em tempo real.
- Sensor de temperatura e umidade (HTS221)
- Sensor de pressão e temperatura (LPS22HB)
- Sensor de movimento de 9 eixos (LSM9DS1: acelerômetro, giroscópio e magnetômetro)

- Sensor de luminosidade (APDS-9960) – também oferece detecção de gestos e cor.

Apesar da variedade de sensores disponíveis, neste projeto foi utilizado apenas o microfone digital integrado à placa, uma vez que o foco está na aquisição e classificação de sinais de áudio.

A placa também conta com conectividade Bluetooth Low Energy (BLE), alimentação via USB e compatibilidade direta com a plataforma Edge Impulse, o que facilita a coleta de dados, treinamento do modelo e deploy direto para o dispositivo.

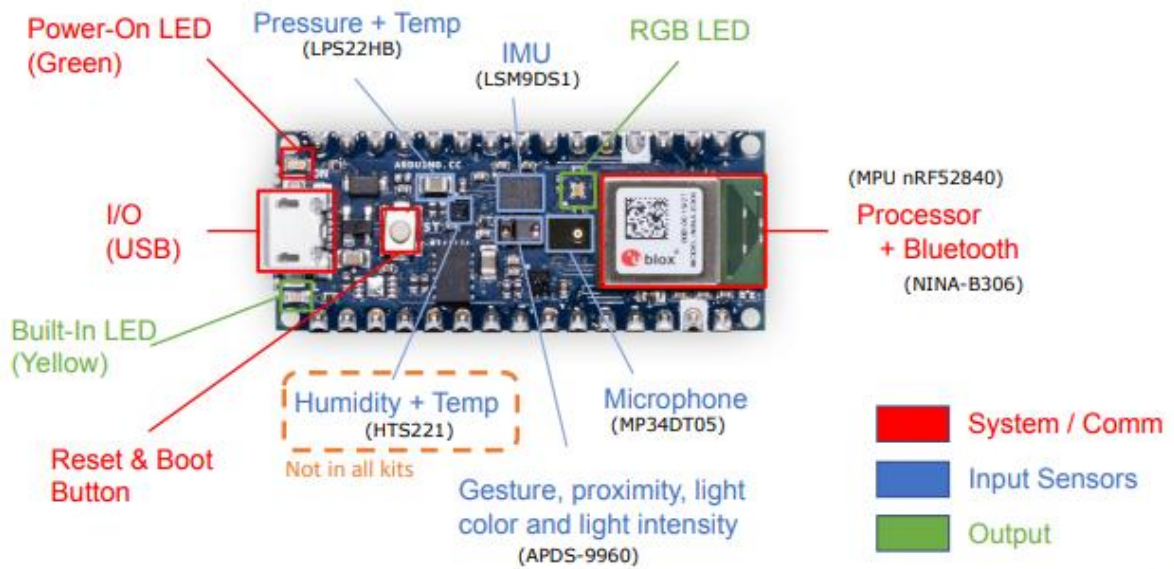


Figura 2: Placa de desenvolvimento

3.2 Aquisição e Preparação de Dados para Modelo de Rede Neural

A construção de um modelo de aprendizado de máquina eficaz depende intrinsecamente da qualidade e da estrutura do seu dataset. Para este projeto, cujo objetivo é classificar sons em três categorias distintas, foi realizado um cuidadoso processo de coleta e preparação de dados, garantindo que o modelo recebesse informações consistentes e relevantes para o treinamento.

3.2.1 Coleta de Dados

A primeira etapa consistiu em obter os dados brutos para cada uma das classes do modelo. As fontes eram distintas para os sons de pássaros e para os ruídos ambientes.

➤ Classes de pássaros: Trinca-Ferro, Tico-Tico e Corvo:

Para as classes de pássaros, os áudios dos cantos do Tico-Tico e do Corvo foram extraídos de vídeos da plataforma YouTube. Foi realizada uma busca por vídeos que continham os cantos de forma clara e com o mínimo de ruído de fundo ou interferência de outros sons. Após a seleção dos vídeos, os áudios foram extraídos para posterior tratamento. Para o Trinca-Ferro, os dados foram obtidos através do Xeno-canto, um dataset com diversos áudios de diferentes aves.

➤ Classe de Ruído Ambiente

Para compor a classe que diferencia os cantos dos pássaros de outros sons, utilizamos o dataset ESC-50 (Environmental Sound Classification). Este é um dataset público e bem estabelecido, contendo 2.000 gravações de curta duração de 50 classes semânticas de sons ambientais, o que garantiu uma grande variedade de ruídos para o treinamento.

3.2.2 Padronização e Processamento de Áudio

Com os dados brutos coletados, a etapa mais crítica foi a de normalização e processamento para garantir a uniformidade do dataset.

Independente da fonte (YouTube ou o dataset ESC-50), todos os arquivos de áudio possuíam características distintas, como taxas de amostragem e durações variadas. Para padronizar todo o conjunto de dados, o mesmo script em Python foi aplicado a todos os arquivos. O script realizou duas tarefas principais:

- 1) Normalização da Frequência: Todos os áudios foram convertidos para uma frequência de amostragem de 16 kHz, um padrão comum em aplicações de reconhecimento de voz e sons.
- 2) Conversão de Formato: Os arquivos foram salvos no formato .wav para garantir a consistência e evitar perdas de qualidade.

Após a normalização, os áudios foram carregados na plataforma Edge Impulse, onde foram segmentados em amostras de 2 segundos, criando entradas de tamanho fixo para o modelo.

3.2.3 A Importância da Curadoria de Dados: Uma Lição Prática

A frase "lixo entra, lixo sai" é especialmente verdadeira no aprendizado de máquina: se os dados usados forem ruins, os resultados também serão ruins. A qualidade do dataset de treinamento é, muitas vezes, mais impactante no resultado final do que a própria arquitetura da rede neural.

A experiência inicial neste projeto ilustra perfeitamente esse ponto. Em uma primeira abordagem, a quantidade de volume massivo de dados era importante no início, totalizando mais de 4 horas de áudio. A hipótese era que uma maior variabilidade nos dados melhoraria a capacidade de inferência e generalização do modelo. Acreditava-se que um dataset mais diversos poderia diminuir a possibilidade de overfitting, mesmo com um número maior de épocas de treinamento. Contudo, com dados mal formatados, todo o esforço foi em vão. Pouca atenção foi dada à qualidade individual de cada amostra, e o resultado foi uma acurácia consistentemente baixa, independentemente das alterações na estrutura da rede neural ou nos hiperparâmetros de treinamento. O modelo estava aprendendo com ruído, áudios corrompidos e amostras irrelevantes.

Diante do resultado insatisfatório, a estratégia foi radicalmente alterada. Em vez de focar na quantidade, foi levado em foco a qualidade. O dataset foi drasticamente reduzido, e cada amostra de áudio foi cuidadosamente auditada. Esse processo de curadoria manual, embora trabalhoso, garantiu que apenas os sons mais claros e representativos de cada classe fossem mantidos. A melhoria foi imediata, com um dataset menor, porém de alta qualidade, o modelo alcançou um desempenho significativamente superior.

Esta experiência prática de construir um dataset do zero representou um aprendizado fundamental. Ao utilizar datasets prontos de repositórios como o Kaggle, muitas vezes não se percebe a complexidade e a importância do trabalho de coleta e tratamento dos dados, pois eles já chegam em um formato ideal. Enfrentar os desafios de lidar com dados brutos, ruidosos e desestruturados nos deu uma apreciação profunda pela etapa de curadoria, revelando-a como uma das fases mais críticas e de maior impacto para o sucesso de um projeto de machine learning.

3.3 Processamento e Estruturação do Dataset

Após a curadoria e a conversão de frequência, os áudios foram carregados na plataforma Edge Impulse. Uma inconsistência notável era a duração variável das amostras. Arquivos de áudio com tempos diferentes podem prejudicar o desempenho do modelo, que espera entradas de tamanho fixo.

Para resolver esse problema, todos os arquivos de áudio, tanto dos pássaros quanto do ESC-50, foram segmentados em amostras de 2 segundos. Esse procedimento resultou em um conjunto de dados homogêneo e pronto para o treinamento.

3.4 Composição Final do Dataset

Ao final do processo de coleta e tratamento, o dataset completo e curado totalizou 10 minutos de áudio. Este foi dividido na proporção de 79/21:

- Dados de Treinamento: 8 minutos
- Dados de Teste: 2 minutos.

A distribuição dos dados foi feita de forma perfeitamente balanceada entre as três classes, garantindo que o modelo não tivesse viés por nenhuma categoria:

- Para treinamento: Aproximadamente 2 minutos de áudio para cada classe.
- Para teste: 30 segundos de áudio para cada classe.

Essa estruturação rigorosa dos dados, validada pela experiência prática, é um passo fundamental para assegurar a criação de um modelo de classificação robusto e preciso.

3.5 Pré processamento

Para extrair informações relevantes dos sinais de áudio capturados, foi utilizado o método de extração de características Mel-Frequency Energy (MFE), no bloco de features no momento da criação do impulso na plataforma Edge Impulse. Essa técnica é semelhante ao Mel-Frequency Cepstral Coefficients (MFCC), ambos baseados na escala mel, que aproxima a percepção humana de frequência. A principal diferença entre eles está no tratamento do espectro: enquanto o MFCC aplica uma transformação adicional (a DCT – Transformada Discreta do Cosseno) sobre as energias filtradas para reduzir a dimensionalidade e eliminar

correlações, o MFE mantém diretamente as energias dos filtros Mel como vetor de características.

Essa diferença torna o MFE mais simples computacionalmente e, em muitos casos, mais eficaz para sons não vocais ou não humanos, como vocalizações de pássaros, onde a estrutura harmônica e os padrões temporais não seguem as mesmas características da fala humana. Por esse motivo, o MFE foi escolhido neste projeto, por oferecer melhor desempenho na representação dos sons das espécies analisadas, além de ser mais adequado para o contexto de execução embarcada com recursos limitados.

The image shows a web interface for configuring MFE parameters. It has a title 'Parameters' and a button 'Autotune parameters'. Below is a section 'Mel-filterbank energy features' with several input fields: 'Frame length' (0.02), 'Frame stride' (0.01), 'Filter number' (40), 'FFT length' (256), 'Low frequency' (0), and 'High frequency' (0). Below this is a 'Normalization' section with a 'Noise floor (dB)' field set to -52. At the bottom right is a 'Save parameters' button.

Parameter	Value
Frame length	0.02
Frame stride	0.01
Filter number	40
FFT length	256
Low frequency	0
High frequency	0
Noise floor (dB)	-52

Figura 3: Parâmetros MFE

3.6 Projeto do modelo

O modelo utilizado no bloco de aprendizado da plataforma Edge Impulse foi o classificador (Classifier), utilizando como entrada as características extraídas via Mel-Frequency Energy (MFE), referentes às quatro classes definidas: Corvo, Tico-Tico, Trinca-Ferro e sons ambientes.

Por padrão, ao criar um classificador na plataforma, o Edge Impulse gera uma arquitetura inicial com os seguintes parâmetros: 8 filtros e 3 neurônios na primeira camada convolucional, e 16 filtros e 3 neurônios na segunda camada convolucional. O modelo também inclui uma camada de Reshape e uma de Flatten, responsáveis por adequar a saída da convolução para a entrada da camada densa final. A Figura 4, ilustra os parâmetros padrão do classificador.

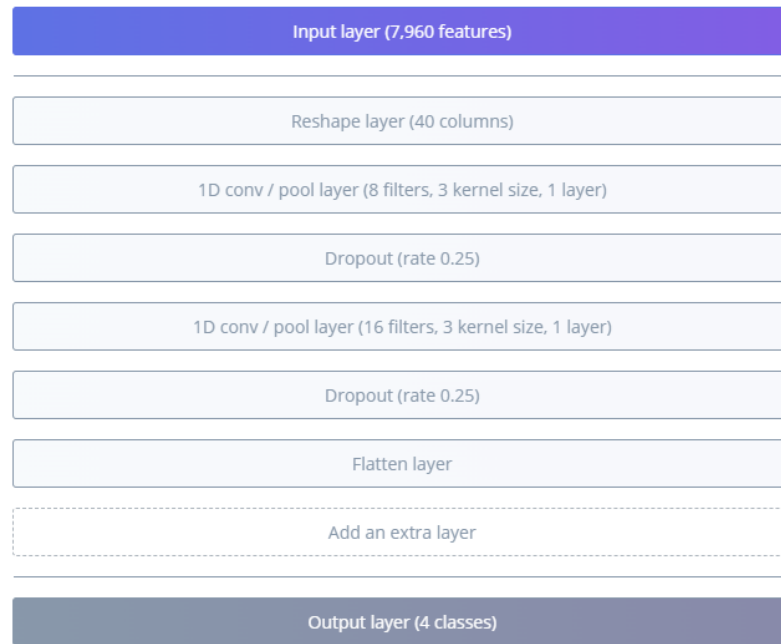


Figura 4: Parâmetros padrão do classificador no Edge Impulse.

Com essa configuração inicial, a acurácia obtida nos testes não ultrapassava os 90%, mesmo com os dados bem organizados e balanceados. Para melhorar o desempenho, foi realizado o ajuste dos hiperparâmetros da rede, aumentando o número de filtros para 12 na primeira camada convolucional e 18 na segunda camada, como ilustrado na Figura 5. Esse ajuste resultou em uma melhoria significativa na capacidade de generalização do modelo, alcançando 97,3% de acurácia no treinamento e 96,72% nos testes.

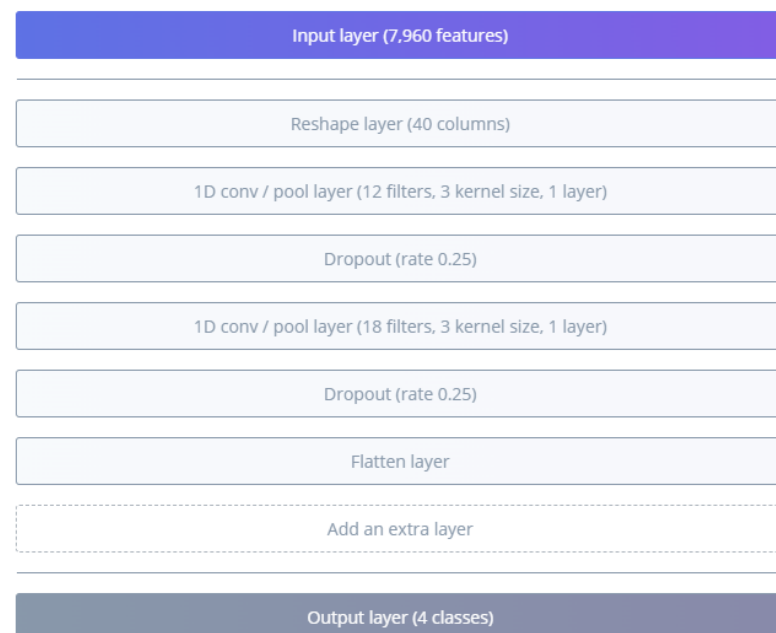


Figura 5: Classificador com o número de filtros modificado.

Apesar do pequeno aumento no uso de memória RAM devido ao aumento no número

de filtros, o consumo final foi de 35,8 KB, valor bastante reduzido em comparação à capacidade total de 256 KB da placa Arduino Nano 33 BLE Sense, mantendo o modelo adequado para execução embarcada com folga de recursos.

3.7 Inferência

A inferência do modelo, otimizada para implementação em dispositivos embarcados, será realizada no Arduino Nano 33 BLE Sense utilizando o modelo de otimização TensorFlow Lite na plataforma do Edge Impulse. O modelo foi quantizado (int8) para diminuir a alocação de memória RAM no Arduino, mesmo diminuindo o número de bits na quantização, ainda foi possível obter uma taxa de acurácia com 96,72% na classificação.

Em relação a latência, foi obtido uma latência de 506 ms para a classificação utilizando MFE, e 25 ms para o classificador, com isso a latência total obtida pelo dispositivo foi de 531 ms, para essa aplicação é uma latência dentro dos limites, visto que não é uma aplicação crítica.

Após concluir todo o processo de treinamento do classificador de sons de pássaros na plataforma online, prosseguimos com a etapa de deploy no dispositivo. A sinalização da classificação das diferentes classes será realizada de forma visual, por meio dos LEDs disponíveis no dispositivo. O LED RGB será utilizado para identificar três espécies de pássaros: vermelho para corvo, verde para trinca-ferro e azul para tico-tico. Já o LED interno (BUILTIN) será ativado para indicar a detecção de sons de ambiente, garantindo uma resposta rápida e clara quanto à classe sonora identificada.

Para integrar o modelo de classificação de sons ao Arduino e fornecer um retorno visual ao usuário, foram desenvolvidas funções específicas para controle dos LEDs do dispositivo. Essas funções são responsáveis por garantir que os LEDs estejam desligados inicialmente, e posteriormente acender o LED correspondente à classe detectada conforme a maior porcentagem na classe obtida no loop principal da aplicação.

A função `Turn_off_leds` é responsável por desligar todos os LEDs no início da aplicação, garantindo que nenhum LED permaneça aceso indevidamente antes do início das classificações. Essa etapa é essencial para evitar interpretações visuais incorretas logo após a inicialização do sistema, assegurando que os LEDs reflitam apenas os resultados reais da inferência sonora. Como os LEDs RGB do Arduino Nano 33 BLE são ativos em nível lógico baixo, a função utiliza `digital_Write` com valor `HIGH` para garantir que todos estejam apagados,

apenas o BUILTIN é apagado com o valor LOW.

```
void turn_off_leds() {
    digitalWrite(LED_R, HIGH);
    digitalWrite(LED_G, HIGH);
    digitalWrite(LED_B, HIGH);
    digitalWrite(LED_BUILTIN, LOW);
}
```

Figura 6: função *Turn_off_leds*

A função `light_led_by_index` é responsável por acender o LED correspondente à classe identificada com maior probabilidade no loop principal. Ela recebe como parâmetro o índice da classificação com maior valor e, com base nesse índice, utiliza uma estrutura `switch/case` para acionar o LED apropriado.

```
void light_led_by_index(int index) {
    turn_off_leds();

    switch (index) {
        case 0:
            digitalWrite(LED_R, LOW); // vermelha
            break;
        case 1:
            digitalWrite(LED_BUILTIN, HIGH); //
            break;
        case 2:
            digitalWrite(LED_B, LOW); // azul
            break;
        case 3:
            digitalWrite(LED_G, LOW); // verde
            break;
        default:
            // nenhum LED acende
            break;
    }
}
```

Figura 7: função *light_led_by_index*

No loop principal de classificação, se encontra um trecho de código que identifica a classe com maior probabilidade retornada pelo modelo. Inicialmente, considera-se a primeira classe (índice 0) como a mais provável. Em seguida, um laço percorre todas as classes e compara seus valores de probabilidade. Se encontrar um valor maior, atualiza-se o índice (`max_index`) da classe correspondente. Ao final, esse índice é passado para a função `light_led_by_index()`, que acende o LED referente à classe com maior chance de ter sido detectada, fornecendo assim um retorno visual direto na placa de desenvolvimento.

```
int max_index = 0;
float max_value = result.classification[0].value;

for (size_t ix = 1; ix < EI_CLASSIFIER_LABEL_COUNT; ix++) {
    if (result.classification[ix].value > max_value) {
        max_value = result.classification[ix].value;
        max_index = ix;
    }
}
light_led_by_index(max_index);

#if EI_CLASSIFIER_HAS_ANOMALY == 1
    ei_printf("    anomaly score: %.3f\n", result.anomaly);
#endif
}
```

Figura 8: Utilização da função `light_led_by_index`

4 CONCLUSÃO

Em suma, este projeto demonstrou com sucesso a viabilidade da aplicação de aprendizado de máquina embarcado (Tiny ML) para a classificação de sons de pássaros em um dispositivo de baixo custo e consumo energético, o Arduino Nano 33 BLE Sense. A implementação do sistema permitiu a distinção de três espécies de pássaros (Corvo, Tico-Tico, Trinca-Ferro) e ruídos ambientes, com indicação visual por meio dos LEDs do dispositivo.

Um dos principais aprendizados práticos foi a importância crítica da curadoria de dados. A experiência de lidar com um volume massivo de dados mal formatados inicialmente resultou em baixa acurácia, enquanto um dataset menor, porém cuidadosamente auditado e de alta qualidade, levou a uma melhoria imediata e substancial no desempenho do modelo. Isso ressalta que a qualidade do dataset de treinamento é frequentemente mais impactante do que a arquitetura da rede neural em si.

O modelo de rede neural convolucional, utilizando a extração de características Mel-Frequency Energy (MFE), alcançou uma alta acurácia de 97,3% no treinamento e 96,72% nos testes, confirmando sua boa capacidade de generalização. A otimização do modelo com Tensor Flow Lite para o Arduino Nano 33 BLE Sense resultou em um consumo de memória RAM de apenas 35,8 KB e uma latência total de 531 ms, o que o torna perfeitamente adequado para aplicações embarcadas em tempo real.

Portanto, o projeto validou a aplicação prática dos conceitos da disciplina estudados na disciplina, superando desafios na preparação de dados e demonstrando que dispositivos embarcados de baixo custo podem executar tarefas complexas de classificação sonora com alta precisão e eficiência.

REFERÊNCIAS

Piczak, K. (2015). ESC-50: Dataset for Environmental Sound Classification. Disponível em: <https://github.com/karolpiczak/ESC-50>.

Xeno-canto. Sharing bird sounds from around the world. Disponível em: <https://xeno-canto.org/>.

Espetacular canto do tico-tico, Disponível em: https://www.youtube.com/watch?v=hkfkDd_eJZY&t=5s

Documentação Microcontrolador: [Nano 33 BLE sense](#)

APÊNDICE A – CONTEÚDOS DO PROJETO

Link para acesso ao vídeo explicativo do projeto:

https://drive.google.com/file/d/14kwhWk3oV38hwHSXr-qRPbJz4OymLGf/view?usp=drive_link

Link para acesso no Edge Impulse: <https://studio.edgeimpulse.com/studio/730160>